# Chat → workflows → graphs

`LlmAgent`

### Chat agent

One model, one instruction, a bag of tools. The loop is "think → call tools → answer".

Breaks when the procedure is long, when you mix hard rules with reasoning, or when context outgrows the window.

`Sequential · Loop · Parallel`

### Template workflows

Deterministic orchestration of sub-agents: always A then B, or fan-out, or loop until a condition.

ADK 2.0 still ships them, but graphs are the flexible replacement. We nested Sequential + Loop, then threw it away.

`Workflow(edges=[…])`

### Graph

Nodes are LLM agents, FunctionNodes, or HITL. Edges carry `Event(route=…)`. State is typed; routing is code.

This e2e-agent is a graph: pin context, bounded investigator, verifier, warehouse write, then pause.

> Prompt-only procedures rot. Put the control plane in the graph: branch on trigger, exit on budget, never let an LLM decide whether a human is allowed to see an unverified claim.

# HITL and token budgets

`RequestInput + ResumabilityConfig`

### Pause without burning compute

- `dormancy_gate` posts the report, then yields `RequestInput`.
- `ResumabilityConfig(is_resumable=True)` — session sleeps; the engineer can reply hours later.
- Validate the resume yourself. `response_schema` is documentation; ADK does not coerce the human.
- Fresh `interrupt_id` per re-prompt, or the loop wedges on the old interrupt.

`TokenBudget in AgentState`

### Bound the run in code, not hope

- Turn cap is a *routed* exit: `budget_exceeded` → verifier → human, never a crash.
- `ADK_MAX_LLM_CALLS` is the whole-invocation breaker: one node turn can hide many Gemini calls.
- 429s on Dynamic Shared Quota take the same honest banner path.
- `tokens_used` lives on the schema; wiring the increment is still a follow-on. Turns are what we enforce today.

> Slide 10 promise, implemented: “strictly bound time and token execution” — the engineer knows when the agent is done, even if the answer is “inconclusive, here is the evidence”.

# Typical flow — and the tool-use gaps

`trigger_router` → `pin_run_context` → `investigator_loop` → `verifier_linter` → `record_outcome` → `dormancy_gate` → `deep_dive`

LLM nodes reason. Everything else is code.

| | |
|---|---|
| **Dumping logs** | A 1.1 MB job log never enters the prompt. Tools return addresses, windows, summaries. Raw bytes live in Artifacts. |
| **Serial tools** | One tool per model turn burns the budget on round-trips. Independent calls belong in the same response so ADK can run them together. |
| **Fake control** | “Strip tools from a downstream node” is not an ADK API. We set `has_pr` and deny in `before_tool_callback`. |
| **Trusting citations** | The model can cite a real artifact that does not contain the claim. Verifier is a FunctionNode: substring check, known SHAs, baseline conjunction. |
| **Sync tools** | A synchronous tool in a batch blocks the rest. Parallel execution needs `async def` (ADK ≥ 1.10). |

# Useful pattern: parallel tool calls

Not this agent's SQL path — useful in others

## One model turn, several queries

When an LLM must search a structured store, sequential "query → empty → rewrite" pays a full reasoning round-trip per miss.

Instruct the model to emit 2–3 alternative queries **in the first response**. ADK runs every function call from that response concurrently. Wall clock ≈ slowest query, not the sum.

| | |
|---|---|
| EXACT | `WHERE address = '580 Moraga Rd'` |
| FUZZY | `WHERE address LIKE '%580 Moraga%'` |
| BROAD | related object / relaxed predicates |

Retry a second parallel batch only if the whole first batch fails. Deduplicate hits. Async tools (`aiohttp` / async SQL) or you still serialise.

Where it applies at Ping

## SQL / SOQL agents

Identity inventory, ticket search, environment catalogues — anywhere recall beats a single brittle query.

e2e-agent does **not** generate SQL. BigQuery is a deterministic `MERGE` in `record_outcome`. Parallelism here is log + outline + changed-files in one investigator turn — same ADK mechanism, different payload.

Article 1 — HatchWorks, Apr 2026
hatchworks.com/blog/.../google-adk-best-practices

Article 2 — ADK docs
adk.dev/tools-custom/performance
"3 tools × 2s → ~2s, not 6s"